



FOR JAVA DEVELOPERS MOVING TO NESTJS

500 or Process Exit?

Spring Boot to NestJS, episode 8: exception filters and errors

Inside a request, an error becomes a response. A promise nobody owns never reaches a filter.

An exception thrown inside a request becomes a response on both Spring and NestJS. A rejected promise that nobody awaited or caught has already left the request, so no filter can see it.

At the time of recording, on Node 22, that unhandled rejection ends the process by default. That is Node's default behaviour, not NestJS exception handling.

Versions on this sheet were current on 30 September 2026: Node 22, NestJS 12, Spring Boot 4.1.1.

In scope

- What each stack returns for a plain, a framework and a domain exception
- Translating a domain exception with a NestJS filter and a Spring advice
- Why a detached rejected promise exits the process, and the two honest fixes

Not in scope

- Process-level hooks: last-resort logging only, never the fix
- What runs around every request: next episode, middleware and interceptors

The Spring bodies shown are this app's default error body on Boot 4.1.1.

01 The measurement

PLAIN ERROR	NestJS 500 generic / Spring 500 generic
NOT FOUND	NestJS 404, message kept / Spring 404, message dropped
DOMAIN, UNTRANSLATED	NestJS 500 / Spring 500, both generic
DOMAIN, TRANSLATED	both 409 { "error": "insufficient_funds", "shortfallInMinorUnits": 250, "currency": "USD" }
NO OWNER	the process EXITED, code 1 (Node v22.22.2)
.catch()	200, failure logged, next request 200
AWAITED	409 from the same filter, next request 200

Both apps with no translation, then with a filter or an advice.

02 The NestJS filter

The domain exception stays free of HTTP. The filter translates it at the edge: name the type, choose the status, choose the body. 409 because the request is well formed but the account's current state prevents it.

```
nestjs-api/src/ep07-errors/filtered.ts
```

```
@Catch(InsufficientFunds)
class InsufficientFundsFilter implements ExceptionFilter<InsufficientFunds> {
  catch(exception: InsufficientFunds, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    // 409: the account's current state prevents it. Not 402, still reserved.
    res.status(HttpStatus.CONFLICT).json({
      error: 'insufficient_funds',
      shortfallInMinorUnits: exception.shortfall,
      currency: exception.currency,
    });
  }
}

app.useGlobalFilters(new InsufficientFundsFilter());
```

An unrecognised exception returns {"statusCode": 500, "message": "Internal server error"}

NestJS documentation, Exception filters, checked 2026-09-30.

402 is reserved for future use; 409 is a conflict with the current state of the target resource

RFC 9110, sections 15.5.3 and 15.5.10.

03 The Spring equivalent

This one transfers almost exactly. The difference is where it lives: Spring finds the advice by component scanning; the NestJS filter is registered in the bootstrap.

```
● ● ● Ep07Advice.java

@RestControllerAdvice
class Ep07Advice {

    record Body(String error, long shortfallInMinorUnits, String currency) {}

    @ExceptionHandler(Ep07ThrowController.InsufficientFunds.class)
    ResponseEntity<Body> onInsufficientFunds(Ep07ThrowController.InsufficientFunds ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(new Body("insufficient_funds", ex.shortfall(), "USD"));
    }
}
```

Both answered 409 with the same body

Run on a real machine, 2026-09-30, by `scripts/verify-errors.sh` in the course repository.

04 Give every promise an owner

Three versions of a route, one line different. With no owner the process exits. Deliberately detached work gets a catch with a real failure policy. Work that belongs to the request is awaited, so its rejection reaches the filter.

```
● ● ● The three routes, each line annotated with its measured outcome

writeAudit(); // exits the process

void writeAudit().catch((err: Error) => { // logged, process up
    logger.error(`background audit write failed: ${err.message}`);
});

await debitAccount(); // 409 from the filter
```

Default unhandled rejection mode is throw; changed from a warning in Node 15.0.0

Node.js CLI documentation, `--unhandled-rejections`, checked 2026-09-30.

Before you ship a NestJS endpoint

1. Keep domain exceptions free of HTTP
2. Translate each one a client can act on with a filter
3. Choose a status with a meaning: 409 for a state conflict, not 402
4. Await work that belongs to the request
5. Give deliberately detached work a catch that does something
6. Never treat a process hook as the fix

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-errors.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS: Exception filters https://docs.nestjs.com/exception-filters

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.